THE SUMMIT PROJECTS

P6 Production Kubernetes

Project 6 of 9 | Uses levels: Levels 8, 9, 16, 17 | ~16 hours

1 Goal and outcome

Turn the demo cluster from Project 5 into something you would let a team depend on. You add what a basic cluster lacks: **TLS** on the Ingress, **autoscaling**, **least-privilege RBAC**, **secrets that can live in Git safely**, **default-deny NetworkPolicies**, **enforced Pod security**, a **Helm chart** replacing raw YAML, and **GitOps with Argo CD** so that the cluster pulls its desired state from Git and CI needs no cluster credentials.

The depth for each part is in the level capstones: Level 9 (Helm) Capstone 9, Level 16 (hardening) Capstone 16 and Level 17 (GitOps) Capstone 17. This project integrates them around **one application and one delivery path**, so you see how they interact.

You finish with:

- a `shop-gitops` repository holding the Helm chart and per-environment values (`dev`, `prod`), the single source of truth for the cluster;
- Argo CD syncing both environments automatically, with prune and self-heal, and promotion by pull request;
- HTTPS on `https://shop.example.com` (a self-signed issuer in the lab; Let's Encrypt on a real domain);
- an HPA scaling the API from 2 to 6 replicas under load;
- RBAC giving a `shop-devs` group read-only access (no Secrets), enforced and tested with `kubectl auth can-i`;
- a sealed secret in Git instead of a plaintext Secret, and Pod Security `restricted` enforced in the `shop` namespace;
- NetworkPolicies proving only the intended flows work.

2 Architecture

What changed from Project 5: delivery moves from "CI runs `kubectl apply`" to "CI updates Git and Argo CD reconciles" (NEW), and the cluster gains security and scaling controls

(NEW).

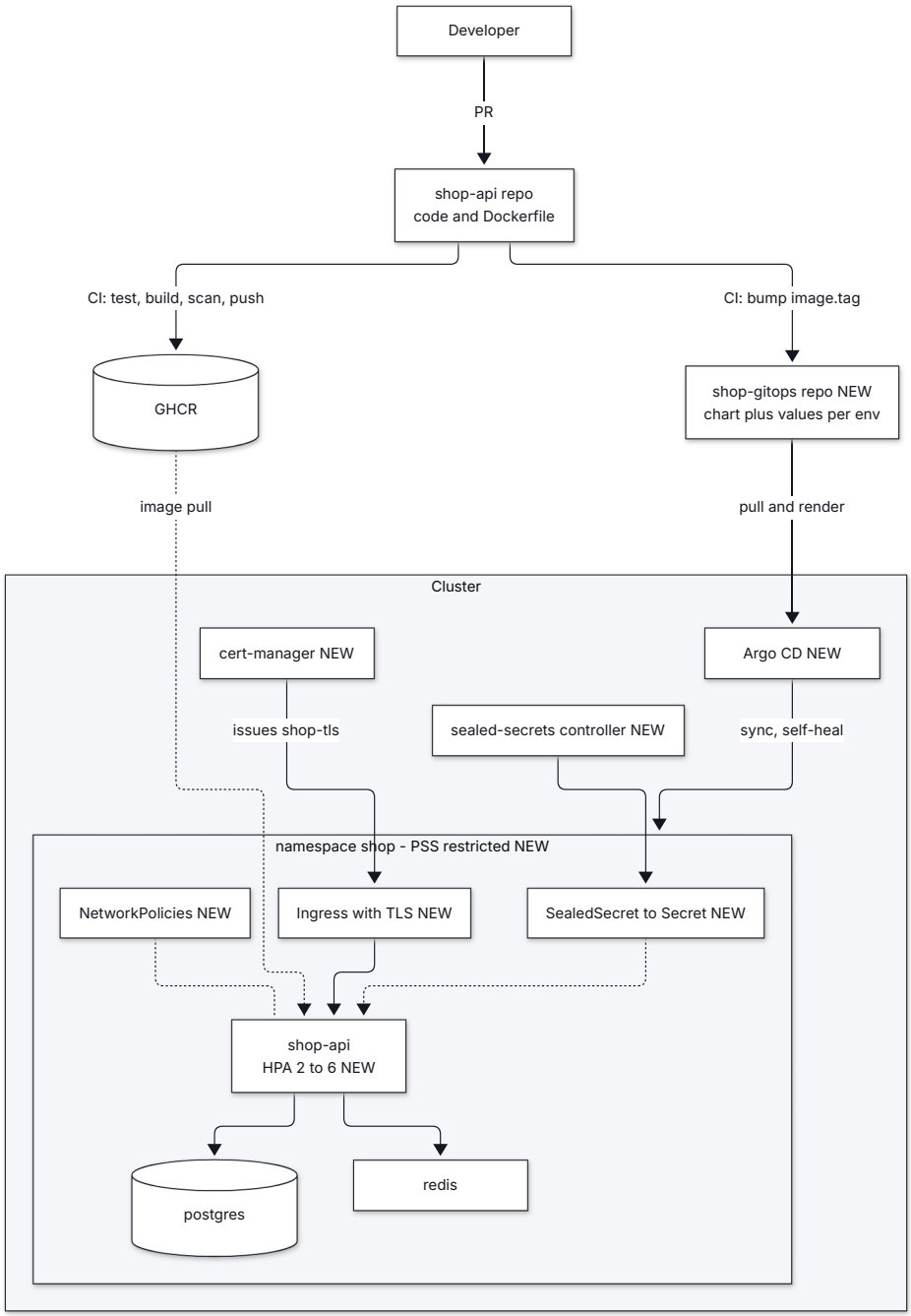


Figure 6-1

Network policy intent (anything not drawn is denied):

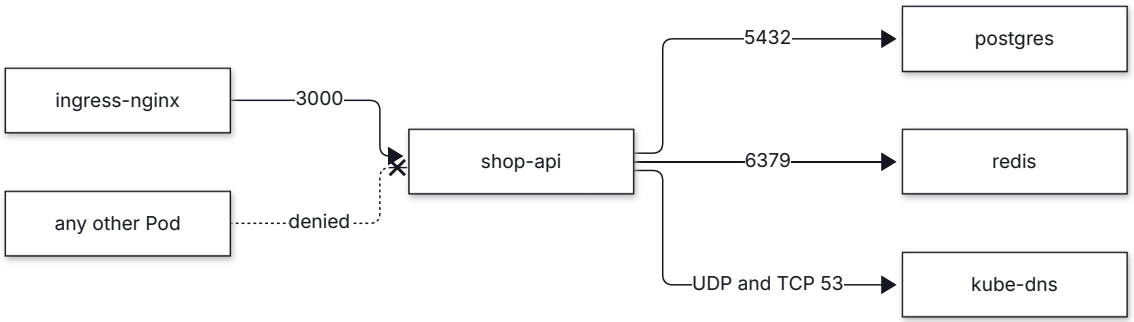


Figure 6-2

Project 5	Project 6
Raw YAML applied by CI	Helm chart in Git, reconciled by Argo CD
CI holds cluster credentials	CI only pushes to Git; the cluster pulls
HTTP only	TLS from cert-manager
Fixed `replicas: 2`	HPA on CPU, plus a PodDisruptionBudget
Secret created by hand	SealedSecret committed to Git
Any Pod talks to any Pod	Default-deny plus explicit allows
`baseline` Pod security	`restricted` enforced
Everyone is admin	Namespaced Roles and a group binding

3 Prerequisites

- A working Project 5 cluster and image (kind is fine, 8 GB RAM; about 4 CPUs). Tools: `kubectl`, `helm`, `kubeseal`, `git`, `gh`, optionally the `argocd` CLI. Cost: nothing locally. On EKS, add the cost of nodes and the control plane, and destroy when done.
- Kubernetes core: RBAC §8.18, NetworkPolicies §8.19, HPA §8.16, Ingress §8.5, Secrets §8.7.
- Helm: charts §9.1, values §9.2, templates §9.3, releases §9.4, packaging applications §9.6.
- Advanced Kubernetes: RBAC in depth §16.1, NetworkPolicies in depth §16.2, Pod security §16.3, HPA, VPA and cluster autoscaler §16.7.
- GitOps: desired state §17.1, Argo CD and Flux §17.2, environment promotion §17.4, CI to GitOps flow §17.7, rollback through Git §17.6.
- Secrets: secrets management §11.4.

4 Build it step by step

Milestone 1 — The Helm chart and the GitOps repository (≈3 h)

Create the `shop-gitops` repository. Turn the Project 5 manifests into a chart, as in the Level 9 capstone [Capstone 9]:

```
shop-gitops/
  charts/shop-api/
    Chart.yaml            # name: shop-api, version: 0.1.0, appVersion from CI
    values.yaml           # safe defaults, every key commented
    values.schema.json    # rejects typos and bad types
    templates/            # deployment, service, ingress, hpa, pdb, networkpolicy,
  serviceaccount, _helpers.tpl
  envs/
    dev/values.yaml
    prod/values.yaml
  apps/                   # Argo CD Application manifests (Milestone 5)
```

`charts/shop-api/values.yaml` (excerpt):

```yaml
image:
  repository: ghcr.io/you/shop-api
  tag: ""                       # set per environment by CI, never "latest"
replicaCount: 2
service: { port: 3000 }
ingress:
  enabled: true
  className: nginx
  host: shop.example.com
  tls: { enabled: true, issuer: lab-selfsigned }
autoscaling: { enabled: true, minReplicas: 2, maxReplicas: 6,
  targetCPUUtilizationPercentage: 70 }
resources:
  requests: { cpu: 100m, memory: 128Mi }
  limits: { memory: 256Mi }
existingSecret: shop-db         # created by the SealedSecret
```

`envs/dev/values.yaml` sets a lower `maxReplicas` and `image.tag`; `envs/prod/values.yaml` sets bigger limits and `replicaCount: 3`. Check rendering and quality before anything touches the cluster:

```bash
helm lint charts/shop-api -f envs/dev/values.yaml
helm template shop charts/shop-api -f envs/dev/values.yaml | kubectl apply --dry-
  run=server -f -
```

Make the Deployment restart when its ConfigMap changes, using a checksum annotation, and put `securityContext` (non-root, read-only root file system, `drop: [ALL]`, `seccompProfile: RuntimeDefault`) in the chart so it is not optional.

Milestone 2 — TLS with cert-manager (≈1 h)

```bash
helm repo add jetstack https://charts.jetstack.io && helm repo update
helm install cert-manager jetstack/cert-manager -n cert-manager --create-namespace --set
  crds.enabled=true
```

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: lab-selfsigned }
spec: { selfSigned: {} }
```

The chart's Ingress adds `cert-manager.io/cluster-issuer: {{ .Values.ingress.tls.issuer }}` and a `tls:` block with `secretName: shop-tls`.

```bash
kubectl -n shop get certificate shop-tls                        # READY True
curl -ks --resolve shop.example.com:8443:127.0.0.1 https://shop.example.com:8443/readyz
```

`-k` is needed only because the lab issuer is self-signed. On a real domain switch the issuer to an ACME `ClusterIssuer` for Let's Encrypt (HTTP-01 or DNS-01) and remove `-k` (TLS §12.4).

Milestone 3 — Autoscaling (≈45 min)

Install metrics-server (on kind add `--kubelet-insecure-tls`, lab only). The chart's `hpa.yaml`:

```yaml
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: {{ include "shop-api.fullname" . }} }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: {{ include "shop-
  api.fullname" . }} }
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: {{
  .Values.autoscaling.targetCPUUtilizationPercentage }} }
{{- end }}
```

When the HPA is enabled, remove `replicas:` from the Deployment template; otherwise every sync would reset the count Argo CD sees. Add a PodDisruptionBudget with `minAvailable: 1` so a node drain cannot take every replica at once.

Milestone 4 — RBAC, Pod security, secrets and NetworkPolicies (≈3 h)

RBAC. A read-only role for developers, without Secrets:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: shop-viewer, namespace: shop }
rules:
  - apiGroups: [""]
    resources: [pods, pods/log, services, endpoints, configmaps, events]
    verbs: [get, list, watch]
  - apiGroups: [apps]
    resources: [deployments, replicasets, statefulsets]
    verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: shop-devs-view, namespace: shop }
subjects: [{ kind: Group, name: shop-devs, apiGroup: rbac.authorization.k8s.io }]
roleRef: { kind: Role, name: shop-viewer, apiGroup: rbac.authorization.k8s.io }
```

```bash
kubectl auth can-i list pods    -n shop --as=jane --as-group=shop-devs     # yes
kubectl auth can-i get secrets  -n shop --as=jane --as-group=shop-devs     # no
kubectl auth can-i delete pods  -n shop --as=jane --as-group=shop-devs     # no
```

Pod security. Label the namespace `pod-security.kubernetes.io/enforce=restricted` and fix what it rejects (PostgreSQL and Redis need `runAsNonRoot`, an explicit `runAsUser`, `allowPrivilegeEscalation: false`, `drop: [ALL]`, and `seccompProfile`). Read the warnings; they tell you exactly which field is missing (Pod security §16.3).

Sealed secrets. Install the Sealed Secrets controller, then seal the existing secret so that the encrypted form is safe to commit:

```bash
kubectl create secret generic shop-db -n shop --dry-run=client -o yaml \
  --from-literal=POSTGRES_USER=shop --from-literal=POSTGRES_PASSWORD="$DB_PASS" \
  --from-literal=DATABASE_URL="postgres://shop:${DB_PASS}@postgres:5432/shop" \
  | kubeseal --format yaml > envs/dev/sealed-shop-db.yaml
git add envs/dev/sealed-shop-db.yaml
```

Only the controller in that cluster can decrypt it. Back up the controller's key; losing it means re-sealing everything. In a cloud setup, External Secrets Operator reading AWS Secrets Manager (the secret from Project 4) is the usual alternative.

NetworkPolicies. Start with deny-all, then allow only the drawn flows (the chart renders these):

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: shop }
spec: { podSelector: {}, policyTypes: [Ingress, Egress] }
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: api, namespace: shop }
spec:
  podSelector: { matchLabels: { app: shop-api } }
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: ingress-nginx }
  }
      ports: [{ port: 3000 }]
  egress:
    - to: [{ podSelector: { matchLabels: { app: postgres } } }]
      ports: [{ port: 5432 }]
```

```
    - to: [{ podSelector: { matchLabels: { app: redis } } }]
      ports: [{ port: 6379 }]
    - to:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }
          podSelector: { matchLabels: { k8s-app: kube-dns } }
      ports: [{ port: 53, protocol: UDP }, { port: 53, protocol: TCP }]
```

Add matching ingress policies for `postgres` and `redis` that accept traffic only from `app: shop-api`. NetworkPolicies need a CNI that enforces them (Calico or Cilium, as in Project 5); with a CNI that ignores them they silently do nothing.

Milestone 5 — Argo CD and the delivery flow (≈3 h)

BASH

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server --timeout=300s
```

`apps/shop-api-dev.yaml`:

YAML

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: shop-api-dev, namespace: argocd }
spec:
  project: default
  source:
    repoURL: https://github.com/you/shop-gitops.git
    targetRevision: main
    path: charts/shop-api
    helm: { valueFiles: [../../envs/dev/values.yaml] }
  destination: { server: https://kubernetes.default.svc, namespace: shop }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

Define the sealed secret and policies as part of the same repository path (a second source or an `envs/dev` Kustomize layer) so one Application owns the full desired state. For `prod`, remove `automated` (manual sync) or protect the `prod` values with CODEOWNERS (environment promotion §17.4).

Change the application pipeline from Project 5: instead of `kubectl apply`, the `deploy` job commits the new tag to the GitOps repository.

```yaml
      - name: Bump dev image tag in GitOps repo
        env: { GH_TOKEN: "${{ secrets.GITOPS_PAT }}" }
        run: |
          git clone https://x-access-token:${GH_TOKEN}@github.com/you/shop-gitops.git
          cd shop-gitops
          yq -i '.image.tag = "sha-${{ github.sha }}"' envs/dev/values.yaml
          git -c user.name=ci -c user.email=ci@example.com commit -am "shop-api: dev →
  sha-${{ github.sha }}"
          git push
```

Use a fine-grained token that can write only to that one repository. Promotion to `prod` is a pull request that copies the tag from `envs/dev` to `envs/prod`; rollback is `git revert` (roll-back through Git §17.6).

```bash
kubectl -n argocd get applications                          # shop-api-dev  Synced  Healthy
kubectl -n shop delete service shop-api                     # drift on purpose: self-heal
  recreates it within a minute
kubectl -n shop get service shop-api
```

5 Tasks

- [ ] **T1 — Package the app as a chart** *(≈2 h, intermediate)*
  - **Goal:** the Project 5 YAML becomes a parameterised, tested chart.
  - **Steps:**
    1. Create `charts/shop-api` with deployment, service, ingress, hpa, pdb templates and a `values.schema.json`.
    2. Run `helm lint` and `helm template` for dev and prod values; diff the two renders.
    3. Install it manually into a scratch namespace and run `helm test` or the smoke test.
  - **Done when:** `helm lint` is clean, the dev and prod diffs show only intended differences, and a bad value (`replicaCount: "two"`) is rejected by the schema.
- [ ] **T2 — Turn on HTTPS** *(≈1 h, intermediate)*
  - **Goal:** the Ingress serves a certificate issued by cert-manager.
  - **Steps:**
    1. Install cert-manager and the `lab-selfsigned` issuer.
    2. Enable `ingress.tls` and sync.

3. Run `openssl s_client -connect localhost:8443 -servername shop.example.com </dev/null | openssl x509 -noout -subject -dates`.

- **Done when:** `kubectl get certificate` shows `READY True` and the command prints a subject for `shop.example.com`.

- [ ] **T3 — Scale under load** *(≈1 h, intermediate)*
  - **Goal:** watch the HPA add and remove replicas.
  - **Steps:**
    1. Install metrics-server and confirm `kubectl top pods -n shop` works.
    2. Generate load (`kubectl run load --rm -it --image=busybox:1.37 -n shop -- sh -c 'while true; do wget -q -O- http://shop-api:3000/products >/dev/null; done'`, adjusting your NetworkPolicy to allow it temporarily).
    3. Watch `kubectl -n shop get hpa -w` and then stop the load.
  - **Done when:** replicas rise above 2 and later fall back to 2, and you captured the HPA events with `kubectl describe hpa`.

- [ ] **T4 — Enforce least privilege with RBAC** *(≈45 min, intermediate)*
  - **Goal:** developers can look but not touch secrets or delete.
  - **Steps:**
    1. Apply the Role and RoleBinding.
    2. Run the three `kubectl auth can-i` checks with `--as` and `--as-group`.
    3. Try `kubectl auth can-i --list -n shop --as=jane --as-group=shop-devs` and explain every line.
  - **Done when:** the checks print `yes`, `no`, `no` as shown, and you can name why `pods/log` is a separate resource.

- [ ] **T5 — Default-deny and prove the policy** *(≈1.5 h, advanced)*
  - **Goal:** only the intended flows work.
  - **Steps:**
    1. Apply the default-deny and allow policies.
    2. From a debug Pod in the `shop` namespace, try `nc -zv postgres 5432` (must fail) and from the API Pod verify the request through the Ingress still works.
    3. Remove the DNS egress rule and observe what breaks, then restore it.
  - **Done when:** direct Pod-to-database from an unlabelled Pod times out, the app still serves `/products`, and you can explain why removing DNS egress broke the API.

- [ ] **T6 — Put the secret in Git safely** *(≈1 h, advanced)*

- **Goal:** no plaintext secret anywhere in the repository history.
- **Steps:**
  1. Install Sealed Secrets, seal `shop-db`, delete the plaintext Secret and let Argo CD recreate it.
  2. Run a secret scanner over the repo (`gitleaks detect`).
  3. Copy the sealed file to a different cluster (or namespace) and observe that it will not decrypt.
- **Done when:** the API runs with a Secret created by the controller, the scanner finds nothing, and the cross-cluster attempt fails.

☐ **T7 — Deliver a change through Git only** *(≈1.5 h, advanced)*

- **Goal:** a merge to the app repo ends up running in the cluster without anyone running `kubectl`.
- **Steps:**
  1. Install the Argo CD Application with automated sync.
  2. Merge an application change; watch the pipeline bump the dev tag and Argo CD sync it.
  3. Open a PR promoting the tag to `prod`; after merge, sync and verify.
- **Done when:** the running image tag in each environment matches the Git values file, and the audit trail is the Git history.

☐ **T8 — Prove it: recover from drift and a bad release** *(≈1.5 h, advanced)*

- **Goal:** demonstrate GitOps self-healing and rollback.
- **Steps:**
  1. Delete the Service and edit the Deployment's image by hand; watch Argo CD revert both.
  2. Merge a release that breaks `/readyz`; observe the rollout stall while the old Pods keep serving.
  3. `git revert` the bad commit and watch the environment recover. Explain who could do each action and why.
- **Done when:** all three recoveries happen without a manual `kubectl apply`, and you can describe the blast radius of a stolen CI token versus a stolen cluster admin credential.

6 Definition of done

- [ ] `argocd` shows both environments `Synced` and `Healthy`; every change since setup came through a Git commit.
- [ ] `https://shop.example.com` works with a certificate managed by cert-manager; HTTP redirects to HTTPS.
- [ ] The HPA scales between 2 and 6 replicas; a PodDisruptionBudget exists; requests and limits are set.
- [ ] `kubectl auth can-i get secrets --as=jane --as-group=shop-devs -n shop` returns `no`; CI holds no cluster credentials.
- [ ] No plaintext secret in Git history; the sealed secret decrypts only in its cluster.
- [ ] Default-deny NetworkPolicies are in place with exactly the intended allows, and the namespace enforces Pod Security `restricted`.
- [ ] A rollback by `git revert` has been rehearsed.

7 Break it on purpose

1. **Forget DNS in the egress policy.** Apply default-deny with no DNS rule; the API cannot resolve `postgres`. Find it with `kubectl logs` and a debug Pod. *(Lesson: deny-all breaks name resolution first.)*
2. **Hand-edit a managed resource.** `kubectl -n shop edit deploy shop-api` and change the image. Argo CD reports `OutOfSync` and, with self-heal, reverts it. Turn self-heal off and see the difference. *(Lesson: Git, not kubectl, is the source of truth.)*
3. **Violate Pod Security.** Add `privileged: true` to a container in a branch. The Pod is rejected at admission with a clear message, and Argo CD shows the sync failed. *(Lesson: policy at admission blocks bad config before it runs.)*
4. **Lose the sealing key.** Delete the Sealed Secrets controller's key Secret in a throwaway cluster and reinstall; existing SealedSecrets stop decrypting. Restore from your backup. *(Lesson: the key is a recovery dependency; back it up.)*

8 How this evolves into the next project

The platform now delivers and protects itself, but you can only guess how it behaves. When the p95 latency doubles, you have `kubectl top` and `kubectl logs`. Project 7 — Observability adds the missing senses: Prometheus metrics and alerts, Grafana dashboards, Loki logs,

OpenTelemetry traces and Sentry error tracking, all deployed through the same GitOps repository so that observability is itself declarative. The NetworkPolicies you wrote will need new allows for scraping, which is the first lesson of the next project.